CSD624

Advanced Database Management

in Microsoft SQL Server

Lecture Notes

Dr A.J. Fofanah

Master of Science in Computer Science

5 Sections • Detailed Theory • Practical SQL Examples • Production Patterns

Table of Contents

Section 1: Query Optimization & Execution Plans

Query optimisation is one of the most critical skills for a database professional. Even well-designed schemas can suffer catastrophic performance degradation when queries are poorly written or the optimiser is given insufficient information to make good decisions. This section covers how SQL Server processes queries internally, how to read and interpret execution plans, and the full range of techniques available to tune query performance in production environments.

1.1 How SQL Server Processes a Query

When you submit a T-SQL statement, SQL Server passes it through a pipeline of stages before any data is touched. Understanding this pipeline is fundamental to diagnosing performance issues.

The four stages are: Parsing, Algebrization, Optimisation, and Execution.

During Parsing, the text is checked for syntax correctness and converted into a parse tree. Algebrization resolves all object names, column references, and data types, producing an algebrized tree of logical operators. Optimisation is where the Query Optimizer takes over — it explores thousands of alternative execution strategies, estimates the cost of each using statistics, and selects the cheapest plan. Finally, the Storage Engine executes the chosen plan, retrieving pages from the buffer pool (in-memory cache) or from disk if not cached.

◆ KEY CONCEPT

The Query Optimizer is a cost-based optimizer — it does not guarantee the fastest plan, only the lowest estimated cost plan. If statistics are stale, the cost estimates are wrong, and the plan choice will be wrong.

SQL Code Example

```sql
-- Stage 1: Check what the parser produces (syntax check only)
SET PARSEONLY ON;
SELECT CustomerID, OrderDate, TotalAmount
FROM Sales.Orders
WHERE CustomerID = 42 AND OrderDate >= '2024-01-01';
SET PARSEONLY OFF;

-- Stage 2-3: Show the logical + physical plan WITHOUT executing
SET SHOWPLAN_XML ON;
GO
SELECT o.OrderID, c.CustomerName, SUM(od.Quantity * od.UnitPrice) AS OrderTotal
FROM Sales.Orders o
JOIN Sales.Customers c  ON c.CustomerID = o.CustomerID
JOIN Sales.OrderDetails od ON od.OrderID = o.OrderID
WHERE o.OrderDate >= '2023-01-01'
  AND o.Status = 'Completed'
GROUP BY o.OrderID, c.CustomerName
HAVING SUM(od.Quantity * od.UnitPrice) > 1000
```

```
ORDER BY OrderTotal DESC;
GO
SET SHOWPLAN_XML OFF;

-- Stage 4: Actual execution with runtime statistics
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
SELECT o.OrderID, c.CustomerName
FROM Sales.Orders o
JOIN Sales.Customers c ON c.CustomerID = o.CustomerID
WHERE o.OrderDate >= '2023-01-01';
-- Output: logical reads, CPU time, elapsed time per table
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```

Code Walkthrough

The SHOWPLAN_XML approach is non-destructive — it does not execute the query. It is safe to use in production for diagnosing expensive queries before running them. STATISTICS IO reveals the number of logical reads (from buffer pool) and physical reads (from disk), which is the most reliable measure of query cost because it is independent of server load.

1.2 Reading Execution Plans

The graphical execution plan in SSMS reads right-to-left, bottom-to-top. Each node is a physical operator. The arrow thickness represents the estimated number of rows flowing between operators — a thick arrow where you expect a thin one (or vice versa) is a red flag indicating a cardinality estimation problem.

Critical operators to recognise include the following. An Index Seek accesses the index at a specific key range and is highly efficient; it is what you always want to see on large tables. An Index Scan reads all or many leaf pages and is acceptable for small tables but a warning sign on large ones. A Table Scan reads every single page in the table and is almost always a problem at scale. A Key Lookup fetches non-indexed columns from the base table after an index seek; each lookup is an extra random I/O, so if lookups are frequent the correct fix is to add a covering index. A Hash Match operator builds a hash table from the smaller input and is used when no supporting index exists; it is memory-intensive and can spill to tempdb. Nested Loops iterates the outer input and performs an index seek into the inner table for each row, which is efficient when the outer input is small. Finally, a Merge Join requires both inputs to be pre-sorted on the join key and is very efficient for large sorted datasets.

◆ PRACTICAL TIP

Always check the estimated vs actual row count on each operator (hover in SSMS). A ratio greater than 10× indicates the optimizer was badly misled — investigate statistics or query structure.

SQL Code Example

```sql
-- Compare join strategies by forcing them (educational use only)
-- Hash Join — good for large unsorted datasets, high memory use
SELECT c.CustomerName, COUNT(o.OrderID) AS OrderCount
FROM Sales.Customers c
LEFT JOIN Sales.Orders o ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerName
OPTION (HASH JOIN, MAXDOP 1);

-- Nested Loop Join — good for small outer input + indexed inner
SELECT c.CustomerName, COUNT(o.OrderID) AS OrderCount
FROM Sales.Customers c
LEFT JOIN Sales.Orders o ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerName
OPTION (LOOP JOIN, MAXDOP 1);

-- Merge Join — requires sorted inputs (usually via index)
SELECT c.CustomerName, COUNT(o.OrderID) AS OrderCount
FROM Sales.Customers c
LEFT JOIN Sales.Orders o ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerName
OPTION (MERGE JOIN, MAXDOP 1);

-- Find the costliest queries in the plan cache
SELECT TOP 10
    qs.total_elapsed_time / qs.execution_count     AS avg_elapsed_us,
    qs.total_logical_reads / qs.execution_count    AS avg_logical_reads,
    qs.total_worker_time  / qs.execution_count     AS avg_cpu_us,
    qs.execution_count,
    SUBSTRING(qt.text,(qs.statement_start_offset/2)+1,200) AS query_text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
ORDER BY avg_elapsed_us DESC;
```

Code Walkthrough

The hint options (HASH JOIN, LOOP JOIN, MERGE JOIN) are powerful for education but should never be left in production code without thorough benchmarking — they prevent the optimizer from adapting as data distributions change. The plan cache DMV query is invaluable for identifying the top offenders to address first.

1.3 Statistics & Cardinality Estimation

Statistics objects in SQL Server store histograms describing the distribution of values in indexed columns. The Query Optimizer reads these histograms to estimate how many rows a predicate will return — a figure called the cardinality estimate. If the estimate is wildly wrong, the optimizer will choose the wrong plan.

Statistics become stale when rows are inserted, updated, or deleted beyond a threshold. By default, SQL Server triggers an auto-update when approximately 20% of rows change (the "500 rows + 20%" rule), but for very large tables this threshold is rarely hit and statistics can remain stale for months.

The FULLSCAN option forces SQL Server to read every row in the table when rebuilding the histogram, producing the most accurate statistics possible. The default sampling rate (around 20-30%) is faster but less accurate for skewed distributions.

◆ EXAM FOCUS

Know the difference between AUTO_UPDATE_STATISTICS (synchronous, blocks query) and AUTO_UPDATE_STATISTICS_ASYNC (background update, query uses old stats once then new stats next execution).

SQL Code Example

```sql
-- Inspect the histogram for a specific index/column
-- DENSITY_VECTOR: uniqueness of column combinations (lower = more unique)
-- HISTOGRAM: step-wise distribution of sampled values
DBCC SHOW_STATISTICS ('Sales.Orders', 'IX_Orders_CustomerID');

-- Update statistics with maximum accuracy (full scan of all rows)
UPDATE STATISTICS Sales.Orders WITH FULLSCAN;

-- Update all tables in the database (maintenance window use)
EXEC sp_updatestats;

-- Enable asynchronous statistics updates (recommended for OLTP)
ALTER DATABASE SalesDB SET AUTO_UPDATE_STATISTICS_ASYNC ON;

-- Identify tables with potentially stale statistics
-- (sampled fewer than 30% of rows, or last updated > 7 days ago)
SELECT
    OBJECT_NAME(s.object_id)                          AS table_name,
    s.name                                             AS stat_name,
    sp.last_updated,
    sp.rows,
    sp.rows_sampled,
    CAST(100.0 * sp.rows_sampled / NULLIF(sp.rows,0) AS DECIMAL(5,2)) AS
pct_sampled,
    sp.modification_counter  -- rows changed since last update
FROM sys.stats s
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
WHERE sp.rows > 10000              -- only tables worth worrying about
  AND (
    sp.pct_sampled < 30            -- low sample rate
    OR sp.last_updated < DATEADD(DAY,-7,GETDATE())  -- older than 7 days
    OR sp.modification_counter > sp.rows * 0.1      -- >10% rows changed
  )
ORDER BY sp.modification_counter DESC;
```

Code Walkthrough

The modification_counter column is the critical field — once this exceeds the auto-update threshold, SQL Server will trigger a statistics update before the next query execution. For tables

with millions of rows, the threshold is rarely hit quickly. Proactive weekly FULLSCAN updates during off-peak hours are recommended for VLDB environments.

1.4 SARGable Predicates

A predicate is SARGable (Search ARGument Able) if SQL Server can use it to limit the range of an index scan or to perform an index seek. Non-SARGable predicates force the engine to evaluate the expression for every row — effectively nullifying the index.

The most common SARGability violations are: 1. Applying a function to the indexed column (YEAR(), MONTH(), CONVERT(), LEFT(), etc.) 2. Using LIKE with a leading wildcard ('%ABC') 3. Performing arithmetic on the indexed column (WHERE Salary * 1.1 > 50000) 4. Implicit data type conversions (comparing an INT column to a VARCHAR value)

The fix is always the same: move the transformation to the right side of the predicate (the literal value), leaving the column bare on the left side.

◆ PERFORMANCE

A single non-SARGable predicate on a table with 10 million rows can turn a 1ms seek into a 45-second full scan. This is the single most impactful quick fix in query tuning.

SQL Code Example

```sql
-- == NON-SARGABLE examples (force full Index Scan / Table Scan) ==

-- ✗ Function on column — cannot seek
SELECT * FROM Sales.Orders WHERE YEAR(OrderDate) = 2024;
SELECT * FROM Sales.Orders WHERE MONTH(OrderDate) IN (1,2,3);
SELECT * FROM Sales.Orders WHERE LEFT(OrderRef, 4) = 'ORD-';
SELECT * FROM Sales.Orders WHERE CONVERT(VARCHAR, OrderID) = '10042';

-- ✗ Arithmetic on column — cannot seek
SELECT * FROM HR.Employees WHERE BaseSalary * 1.12 > 60000;

-- ✗ Implicit conversion (CustomerID is INT, '42' is VARCHAR) — adds
CONVERT_IMPLICIT
SELECT * FROM Sales.Orders WHERE CustomerID = '42';

-- ✗ Leading wildcard — full scan (cannot seek forward in index)
SELECT * FROM Customers WHERE Email LIKE '%@gmail.com';

-- == SARGABLE rewrites ==

-- ✅ Range predicate on the column — allows Index Seek
SELECT * FROM Sales.Orders
WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01';

```

```
-- ✅ Function on the literal — column stays bare
SELECT * FROM HR.Employees WHERE BaseSalary > 60000 / 1.12;

-- ✅ Matching data types
SELECT * FROM Sales.Orders WHERE CustomerID = 42;

-- ✅ Trailing wildcard — can seek on prefix
SELECT * FROM Customers WHERE Email LIKE 'john%';

-- Detect implicit conversions in the plan cache
SELECT DISTINCT SUBSTRING(qt.text,1,200) AS query_text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
WHERE CAST(qp.query_plan AS NVARCHAR(MAX)) LIKE '%CONVERT_IMPLICIT%';
```

Code Walkthrough

The CONVERT_IMPLICIT warning in an execution plan is SQL Server telling you that it had to silently convert one side of a comparison to match data types. This always breaks index seeks on the converted column. The fix is to ensure your application sends parameters with the correct data type, or to explicitly cast the literal.

1.5 Parameter Sniffing — Diagnosis and Resolution

Parameter sniffing is one of the most misunderstood SQL Server performance problems. When SQL Server first compiles a stored procedure or parameterised query, it sniffs the parameter values and builds an execution plan optimised specifically for those values. That plan is then cached and reused for all subsequent executions — even when the data distribution is very different.

Consider a CustomerOrders procedure: if the first call uses a customer with 3 orders, the plan uses nested loops (good for small inputs). If a second call uses a customer with 500,000 orders, the cached nested-loop plan is catastrophically slow.

Diagnosis: look for large gaps between min_logical_reads and max_logical_reads in sys.dm_exec_query_stats. A query that sometimes reads 100 pages and sometimes reads 5,000,000 pages is almost certainly a sniffing victim.

◆ BEST PRACTICE

OPTION(RECOMPILE) is the nuclear option — effective but expensive. For high-frequency procedures, try local variables or OPTIMIZE FOR UNKNOWN first, as they avoid the per-execution compilation overhead.

SQL Code Example

```
-- Diagnose parameter sniffing victims in the plan cache
-- Large gap between min and max reads = sniffing suspect
```

```
SELECT
    qs.execution_count,
    qs.min_logical_reads,
    qs.max_logical_reads,
    qs.max_logical_reads - qs.min_logical_reads  AS read_variance,
    SUBSTRING(qt.text, 1, 300)                   AS query_text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
WHERE qs.max_logical_reads > qs.min_logical_reads * 50   -- 50x variance
  AND qs.execution_count > 10
ORDER BY read_variance DESC;

-- == Solution 1: OPTION(RECOMPILE) — fresh plan every time ==
CREATE PROCEDURE Sales.GetCustomerOrders @CustID INT AS
BEGIN
    SELECT o.OrderID, o.OrderDate, o.TotalAmount
    FROM Sales.Orders o
    WHERE o.CustomerID = @CustID
    ORDER BY o.OrderDate DESC
    OPTION (RECOMPILE);  -- compiles new plan for each @CustID value
END;

-- == Solution 2: Local variable technique ==
-- Breaks sniffing: optimizer cannot see the runtime value of @LocalID
CREATE PROCEDURE Sales.GetCustomerOrders2 @CustID INT AS
BEGIN
    DECLARE @LocalID INT = @CustID;  -- copy to local variable
    SELECT o.OrderID, o.OrderDate, o.TotalAmount
    FROM Sales.Orders o
    WHERE o.CustomerID = @LocalID;
END;

-- == Solution 3: OPTIMIZE FOR UNKNOWN ==
-- Uses column average statistics instead of sniffed value
CREATE PROCEDURE Sales.GetCustomerOrders3 @CustID INT AS
BEGIN
    SELECT o.OrderID, o.OrderDate, o.TotalAmount
    FROM Sales.Orders o
    WHERE o.CustomerID = @CustID
    OPTION (OPTIMIZE FOR (@CustID UNKNOWN));
END;

-- == Solution 4: Plan Guide (when you cannot change the code) ==
EXEC sp_create_plan_guide
    @name      = N'PG_CustomerOrders',
    @stmt      = N'SELECT OrderID,OrderDate,TotalAmount FROM Sales.Orders WHERE
CustomerID=@CustID ORDER BY OrderDate DESC',
    @type      = N'SQL',
    @module_or_batch = NULL,
    @params    = N'@CustID INT',
    @hints     = N'OPTION (OPTIMIZE FOR (@CustID UNKNOWN))';
```

Code Walkthrough

The local variable technique is subtle: because SQL Server cannot see the value of a local variable at compile time, it falls back to average-selectivity statistics, producing a plan that is reasonable for typical parameter values. This is often the best balance between plan quality and compilation overhead for procedures called thousands of times per second.

1.6 Adaptive Query Processing (SQL Server 2019+)

Adaptive Query Processing (AQP), expanded to Intelligent Query Processing (IQP) in SQL Server 2019, allows the optimizer to adjust plans during or between executions based on actual runtime data. This is a fundamental change from the traditional compile-once, execute-forever model.

Key IQP features include four major capabilities. Batch Mode Adaptive Joins choose the join type (Hash Join or Nested Loop) at runtime after the actual input row count has been measured, rather than relying on the compile-time estimate. Memory Grant Feedback corrects over- and under-sized memory grants automatically: if a sort or hash operation spills to tempdb because the grant was too small, subsequent executions receive a larger grant; over-grants are reduced in the same way. Table Variable Deferred Compilation (TVDC) addresses the longstanding problem that table variables were always estimated as having one row regardless of their actual count — with TVDC, the query is compiled at first use with the real cardinality. Interleaved Execution allows multi-statement table-valued functions to pass their actual row count to downstream operators instead of a fixed estimate.

SQL Code Example

```sql
-- Enable IQP features (requires compat level 150 for SQL 2019)
ALTER DATABASE SalesDB SET COMPATIBILITY_LEVEL = 150;

-- Verify which IQP features are active
SELECT name, value_in_use
FROM sys.configurations
WHERE name LIKE '%query%';

-- Detect memory grant spills (before MG Feedback takes effect)
SELECT TOP 20
    qs.execution_count,
    qs.total_spills,
    qs.avg_spills,
    qs.last_spills,
    SUBSTRING(qt.text, 1, 250) AS query_text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
WHERE qs.total_spills > 0
ORDER BY qs.total_spills DESC;

-- Table Variable Deferred Compilation in practice
-- OLD behaviour (compat < 150): always estimates 1 row → wrong join/sort strategy
-- NEW behaviour (compat 150):   reads actual count at first use → accurate plan

DECLARE @RecentOrders TABLE (
```

```
    OrderID     INT,
    CustomerID INT,
    Total       DECIMAL(12,2)
);

INSERT INTO @RecentOrders
SELECT OrderID, CustomerID, TotalAmount
FROM Sales.Orders
WHERE OrderDate >= '2024-01-01';     -- might insert 250,000 rows

-- With TVDC: optimizer now knows there are 250,000 rows — uses Hash Join
-- Without TVDC: optimizer thinks 1 row — uses Nested Loop → disaster
SELECT r.OrderID, c.CustomerName, r.Total
FROM @RecentOrders r
JOIN Sales.Customers c ON c.CustomerID = r.CustomerID
ORDER BY r.Total DESC;

-- Check if batch mode is being used (look for BatchModeOnRowStore)
SELECT * FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
WHERE CAST(qp.query_plan AS NVARCHAR(MAX)) LIKE '%BatchModeOnRowStore%';
```

Code Walkthrough

Batch mode on rowstore (available from SQL 2019) allows columnstore-style batch processing (processing 900 rows per CPU cycle rather than 1) on regular B-tree indexes when the table is large enough. This can improve analytic query performance by 5-10x without requiring a columnstore index. Check the query plan for the BatchModeOnRowStore attribute to confirm it is active.

Section 2: Advanced Indexing Strategies

Indexes are the single most powerful tool for improving query performance in SQL Server. A well-designed index can reduce query time from minutes to milliseconds. Equally, poorly designed indexes — too many, too wide, or on the wrong columns — can cripple write performance and waste significant storage. This section covers the full spectrum of indexing techniques required for master-level database engineering.

2.1 B-Tree Index Internals

Every non-columnstore index in SQL Server is a B-Tree (Balanced Tree) structure. Understanding the internal layout is essential for predicting how index maintenance, fragmentation, and key design affect performance.

A B-Tree index has three page types: the Root page (one page, contains the highest-level key ranges), Intermediate pages (zero or more levels, narrow the search), and Leaf pages (the bottom level, contain either the actual data rows or row locators).

For a Clustered Index, leaf pages contain the actual data rows in physical key order — the table and the index are the same structure. Because of this, only one clustered index can exist per table.

For a Non-Clustered Index, leaf pages contain the index key columns plus the clustering key (for CI tables) or the Row ID (for heaps). Looking up additional columns not in the index requires a Key Lookup — an extra random I/O back to the base table.

Fill Factor controls what percentage of each leaf page is filled during an index rebuild. A fill factor of 80 leaves 20% free space, reducing page splits on subsequent inserts at the cost of additional space and I/O.

◆ DESIGN RULE

Choose a clustered index key that is: (1) narrow — it is repeated in every non-clustered index; (2) unique — duplicates require a 4-byte uniquifier; (3) ever-increasing — random values (e.g. GUID) cause severe fragmentation.

SQL Code Example

```sql
-- Inspect the B-Tree depth and page counts for an index
SELECT
    i.name                          AS index_name,
    i.type_desc,
    ips.index_depth,                -- number of B-Tree levels
    ips.index_level,                -- 0=leaf, 1=intermediate, n=root
    ips.page_count,
    ips.record_count,
    ips.avg_record_size_in_bytes    AS avg_row_bytes,
    ips.avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(
    DB_ID(), OBJECT_ID('Sales.Orders'), NULL, NULL, 'DETAILED') ips
```

```
JOIN sys.indexes i
    ON i.object_id = ips.object_id AND i.index_id = ips.index_id
ORDER BY i.index_id, ips.index_level DESC;

-- Examine a table's index structure in detail
SELECT
    i.index_id,
    i.name                          AS index_name,
    i.type_desc,
    i.is_unique,
    i.fill_factor,
    STRING_AGG(
        CASE WHEN ic.is_included_column = 0
             THEN c.name + ' ' + CASE ic.is_descending_key WHEN 1 THEN 'DESC'
ELSE 'ASC' END
             ELSE NULL END,
        ', ') WITHIN GROUP (ORDER BY ic.key_ordinal) AS key_columns,
    STRING_AGG(
        CASE WHEN ic.is_included_column = 1 THEN c.name ELSE NULL END,
        ', ') WITHIN GROUP (ORDER BY ic.index_column_id) AS included_columns
FROM sys.indexes i
JOIN sys.index_columns ic ON ic.object_id = i.object_id AND ic.index_id =
i.index_id
JOIN sys.columns c         ON c.object_id  = ic.object_id AND c.column_id =
ic.column_id
WHERE i.object_id = OBJECT_ID('Sales.Orders')
GROUP BY i.index_id, i.name, i.type_desc, i.is_unique, i.fill_factor
ORDER BY i.index_id;
```

Code Walkthrough

The index_depth in sys.dm_db_index_physical_stats tells you how many page reads are required to navigate from root to leaf. A depth of 3 means every seek costs 3 logical reads minimum, regardless of how many rows match. For very large tables (billions of rows), even 4-5 levels is normal and acceptable.

2.2 Covering Indexes & Included Columns

A Key Lookup (or RID Lookup for heaps) occurs when an index seek finds the row locator but the query requires columns not in the index. The engine must then fetch those columns from the base table using the row locator — one extra random I/O per row returned. For queries returning thousands of rows, this can mean thousands of extra random I/Os.

A covering index satisfies the entire query — SELECT list, WHERE clause, JOIN conditions, and ORDER BY — entirely from index pages, eliminating the Key Lookup. The INCLUDE clause allows you to add non-key columns to the leaf level only, without extending the B-Tree depth.

The distinction between key columns and included columns is important. Key columns participate in the B-Tree structure, affect the seek depth, and are physically sorted; use them for columns that appear in WHERE clauses, JOIN conditions, and ORDER BY expressions. Included columns are stored only in the leaf pages and do not contribute to the tree depth; they cannot be used to narrow a seek but can be returned without a Key Lookup back to the base

table. Use the INCLUDE clause for SELECT-list columns that are never part of the search predicate.

SQL Code Example

```sql
-- == Demonstrating the Key Lookup problem ==

-- Without covering index: two-step process
-- Step 1: Seek on CustomerID → returns (CustomerID, OrderID) leaf entries
-- Step 2: Key Lookup on each leaf entry → fetches OrderDate, TotalAmount
SELECT CustomerID, OrderDate, TotalAmount
FROM Sales.Orders
WHERE CustomerID = 42;
-- Execution plan: IndexSeek(IX_CustomerID) → KeyLookup(PK_Orders) ← expensive!

-- == Covering index: eliminates Key Lookup entirely ==
CREATE NONCLUSTERED INDEX IX_Orders_Cust_Covering
ON Sales.Orders (CustomerID)          -- seek column: key
INCLUDE (OrderDate, TotalAmount);     -- returned columns: included (leaf-only)
-- Execution plan: IndexSeek only — no Key Lookup!

-- == Multi-column covering index for a complex report ==
-- Query: WHERE SalesRegion = 'EMEA' AND SalesDate BETWEEN ... ORDER BY SalesDate
-- INCLUDE: all columns needed in SELECT list
CREATE NONCLUSTERED INDEX IX_Sales_Region_Date_Cover
ON Sales.Transactions (SalesRegion, SalesDate)  -- equality first, range last
INCLUDE (SalesRepID, ProductID, Revenue, Units, Discount);

-- == Verify the Key Lookup is eliminated ==
-- Before: check sys.dm_db_index_usage_stats for user_lookups
SELECT index_id, user_seeks, user_scans, user_lookups
FROM sys.dm_db_index_usage_stats
WHERE object_id = OBJECT_ID('Sales.Orders')
  AND database_id = DB_ID();
-- user_lookups > 0 means Key Lookups are happening — cover those columns!

-- == Find Key Lookup candidates in the plan cache ==
SELECT SUBSTRING(qt.text,1,300) AS query_text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
WHERE CAST(qp.query_plan AS NVARCHAR(MAX)) LIKE '%KeyLookup%'
   OR CAST(qp.query_plan AS NVARCHAR(MAX)) LIKE '%RIDLookup%';
```

Code Walkthrough

The sys.dm_db_index_usage_stats user_lookups counter is reset on every SQL Server restart, so use it to identify lookup patterns in the current uptime period. An index with millions of user_seeks but also hundreds of thousands of user_lookups is a strong candidate for widening with INCLUDE columns. Always check this before adding new indexes.

2.3 Columnstore Indexes for Analytics

Columnstore indexes store data column by column rather than row by row. This architectural difference produces two major advantages for analytical workloads: extreme compression (5-10x vs rowstore) and batch mode execution (processing ~900 rows per CPU cycle vs 1).

Batch mode execution is the key performance multiplier. In traditional rowstore, every aggregate, sort, or hash operation processes one row at a time. In batch mode, the same operations process vectors of 900 values simultaneously using SIMD CPU instructions, dramatically reducing CPU cycles for aggregation-heavy queries.

Columnstore indexes are ideal for: fact tables in data warehouses, reporting databases, and any table queried with aggregations across millions of rows (GROUP BY, SUM, COUNT, AVG).

A Clustered Columnstore Index (CCI) replaces the entire table storage. A Non-Clustered Columnstore Index (NCCI) is an overlay that can coexist with a rowstore clustered index — enabling hybrid OLTP+Analytics (HTAP) on the same table.

◆ HTAP DESIGN

For tables that need both fast OLTP row operations AND fast analytical aggregations, create a rowstore clustered index (on IDENTITY key) PLUS a non-clustered columnstore index. SQL 2019 supports updatable NCCIs.

SQL Code Example

```
-- == Clustered Columnstore: full table stored as columnstore ==
CREATE TABLE DW.FactSales (
    DateKey      INT          NOT NULL,
    CustomerKey  INT          NOT NULL,
    ProductKey   INT          NOT NULL,
    Quantity     INT,
    UnitPrice    DECIMAL(10,2),
    Discount     DECIMAL(5,4),
    Revenue      DECIMAL(12,2),
    Cost         DECIMAL(12,2)
);

CREATE CLUSTERED COLUMNSTORE INDEX CCI_FactSales
ON DW.FactSales
WITH (MAXDOP = 4, DATA_COMPRESSION = COLUMNSTORE);
-- Result: ~5-10x compression, batch-mode aggregations

-- == Non-Clustered Columnstore: HTAP overlay on OLTP table ==
-- Table has a rowstore clustered index for OLTP writes
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Orders_Analytics
ON Sales.Orders (OrderDate, CustomerID, ProductID, TotalAmount, Status,
SalesRegion)
WITH (MAXDOP = 4);
-- OLTP: rowstore index handles inserts/updates at full speed
-- Analytics: NCCI handles GROUP BY / SUM queries with batch mode

-- == Verify columnstore segment health ==
```

```
SELECT
    OBJECT_NAME(i.object_id)          AS table_name,
    i.name                             AS index_name,
    p.partition_number,
    css.row_count,
    css.size_in_bytes / 1024           AS size_kb,
    css.state_description,             -- OPEN, CLOSED, COMPRESSED
    css.trim_reason_desc               -- why segment was closed early
FROM sys.column_store_segments css
JOIN sys.partitions  p ON p.partition_id  = css.partition_id
JOIN sys.indexes     i ON i.object_id    = p.object_id AND i.index_id =
p.index_id
ORDER BY css.row_count DESC;

-- == Force columnstore segment recompression ==
-- (after large bulk loads, delta rowgroups may be OPEN/uncompressed)
ALTER INDEX CCI_FactSales ON DW.FactSales
REORGANIZE WITH (COMPRESS_ALL_ROW_GROUPS = ON);
```

Code Walkthrough

Columnstore segments aim for 1,048,576 rows (2^20) per row group. Delta row groups (state=OPEN) hold recent inserts and are rowstore until they reach the threshold and get compressed. The REORGANIZE with COMPRESS_ALL_ROW_GROUPS forces compression of all row groups — important after a bulk load to avoid degraded query performance from uncompressed delta stores.

2.4 Index Maintenance: Fragmentation & Rebuild Strategy

Index fragmentation occurs when the logical order of index pages diverges from the physical order (external fragmentation) or when pages are sparsely filled (internal fragmentation). Fragmentation increases the number of I/O operations required to scan an index and can degrade performance significantly for range scans.

Two maintenance operations are available. REORGANIZE defragments leaf pages in-place: it is always online, uses low resources, and is suitable for fragmentation levels between 10 and 30 percent. REBUILD drops and recreates the entire index from scratch, resetting the fill factor; it is offline by default on Standard Edition but can be run with ONLINE=ON on Enterprise Edition, and is the appropriate choice when fragmentation exceeds 30 percent.

After any rebuild, run UPDATE STATISTICS WITH FULLSCAN to ensure the optimizer has fresh histograms.

The Ola Hallengren maintenance scripts are the industry-standard open-source solution for automated, intelligent index maintenance — they check fragmentation level per index and choose REORGANIZE vs REBUILD accordingly.

SQL Code Example

```
-- == Step 1: Measure fragmentation across all significant indexes ==
SELECT
    OBJECT_NAME(ips.object_id)         AS table_name,
    i.name                             AS index_name,
```

```
    ips.index_type_desc,
    ips.avg_fragmentation_in_percent,
    ips.page_count,
    ips.avg_page_space_used_in_percent, -- internal fragmentation
    ips.record_count
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'SAMPLED') ips
JOIN sys.indexes i ON i.object_id = ips.object_id AND i.index_id = ips.index_id
WHERE ips.page_count > 500              -- ignore tiny indexes
  AND ips.index_type_desc <> 'HEAP'
ORDER BY ips.avg_fragmentation_in_percent DESC;

-- == Step 2: Apply the right operation based on fragmentation level ==

-- 10-30% fragmentation: REORGANIZE (always online, low impact)
ALTER INDEX IX_Orders_CustomerID ON Sales.Orders REORGANIZE;

-- > 30% fragmentation: REBUILD (offline by default)
ALTER INDEX IX_Orders_CustomerID ON Sales.Orders
REBUILD WITH (
    FILLFACTOR = 80,     -- 20% free space per leaf page (adjust to write
frequency)
    ONLINE = ON,         -- Enterprise: allow reads/writes during rebuild
    MAXDOP = 2,          -- limit CPU impact
    SORT_IN_TEMPDB = ON -- sort in tempdb, not in the data file
);

-- Rebuild ALL indexes on a table (use during maintenance window)
ALTER INDEX ALL ON Sales.Orders
REBUILD WITH (FILLFACTOR = 85, ONLINE = ON, MAXDOP = 4);

-- == Step 3: Update statistics after rebuild ==
UPDATE STATISTICS Sales.Orders WITH FULLSCAN;

-- == Automated maintenance: choose based on fragmentation level ==
DECLARE @frag FLOAT;
SELECT @frag = avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(DB_ID(), OBJECT_ID('Sales.Orders'), 2, NULL,
'LIMITED');

IF @frag > 30
    ALTER INDEX IX_Orders_CustomerID ON Sales.Orders REBUILD WITH (ONLINE=ON);
ELSE IF @frag > 10
    ALTER INDEX IX_Orders_CustomerID ON Sales.Orders REORGANIZE;
-- else: < 10% — no action needed
```

Code Walkthrough

The SAMPLED mode in sys.dm_db_index_physical_stats is a good balance between accuracy and performance for large databases. DETAILED reads every page and is highly accurate but can take minutes on large tables. LIMITED reads only root and intermediate pages — very fast but only reports fragmentation at the allocation level, missing leaf-level fragmentation.

Section 3: Security, Encryption & Compliance

Database security operates at multiple overlapping layers: authentication (who can connect), authorisation (what they can do), encryption (protecting data at rest and in transit), and auditing (evidencing compliance). For regulated industries — finance, healthcare, government — each layer is mandated by law. GDPR requires demonstrable data protection; HIPAA mandates audit trails; PCI-DSS requires encryption of cardholder data. This section covers the full SQL Server security stack with practical implementation examples.

3.1 Transparent Data Encryption (TDE)

TDE encrypts the entire database at the storage page level using AES-256. Every page written to disk — data files (.mdf), log files (.ldf), and all backup files — is encrypted. The encryption and decryption happen in the buffer pool, transparently to applications and users. No application code changes are required.

The key hierarchy is: Database Encryption Key (DEK, stored encrypted in the database) → protected by a Server Certificate → which is protected by the Service Master Key → which is derived from the Windows DPAPI.

Critical operational risk: if you lose the server certificate and its private key backup, the database is permanently unreadable. Certificate backup must happen immediately after creation and must be stored off-server (secure vault, HSM, or Azure Key Vault).

◆ EXAM CRITICAL

TDE protects data at rest only — it does not protect data in transit (use TLS for that) or data in memory (use Always Encrypted for column-level protection). A logged-in user with SELECT permission can still read all data.

SQL Code Example

```
-- == Full TDE implementation ==

USE master;

-- Step 1: Create the database master key (protects the certificate)
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'Str0ng$ecret!Key2024';

-- Step 2: Create a server certificate for TDE
CREATE CERTIFICATE TDE_Cert_SalesDB
WITH SUBJECT = 'TDE Certificate for SalesDB',
     EXPIRY_DATE = '2029-01-01';

-- Step 3: IMMEDIATELY backup — losing this = losing the database forever
BACKUP CERTIFICATE TDE_Cert_SalesDB
TO FILE = 'E:SecureBackupTDE_Cert_SalesDB.cer'
WITH PRIVATE KEY (
```

```
    FILE       = 'E:SecureBackupTDE_PrivKey_SalesDB.pvk',
    ENCRYPTION BY PASSWORD = 'CertBackup!Key2024'
);

-- Step 4: Create the Database Encryption Key in the target database
USE SalesDB;
CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_256
ENCRYPTION BY SERVER CERTIFICATE TDE_Cert_SalesDB;

-- Step 5: Turn encryption on (background encryption begins)
ALTER DATABASE SalesDB SET ENCRYPTION ON;

-- Step 6: Monitor encryption progress (encryption_state: 3 = encrypted)
SELECT
    d.name,
    dek.encryption_state,
    dek.encryption_state_desc,
    dek.percent_complete,   -- progress during initial encryption
    dek.key_algorithm,
    dek.key_length,
    dek.encryptor_type
FROM sys.dm_database_encryption_keys dek
JOIN sys.databases d ON d.database_id = dek.database_id;
```

Code Walkthrough

The percent_complete field will count from 0 to 100 during the initial encryption scan — SQL Server encrypts all existing pages in the background while the database remains fully online. For a 500 GB database this might take 30-60 minutes. Subsequent backups are automatically encrypted without any additional configuration.

3.2 Always Encrypted

Always Encrypted provides column-level encryption where the SQL Server engine never sees the plaintext — decryption happens exclusively in the client driver (ADO.NET, JDBC, ODBC) using keys stored outside SQL Server. This means even a DBA with sysadmin privileges cannot read the encrypted column values.

Two encryption types are available. Deterministic encryption always produces the same ciphertext for a given plaintext value, which allows equality comparisons in WHERE clauses (for example WHERE SSN = @SSN), but it leaks frequency information to anyone with access to the ciphertext. Randomised encryption produces a different ciphertext each time the same value is encrypted, providing stronger security but making the column unsearchable, unfilterable, and unjoinable. Use deterministic encryption for columns that must be queried, and randomised for columns such as salary figures, medical diagnosis codes, and biometric data that are stored but never used as search predicates.

Always Encrypted Secure Enclaves (SQL Server 2019+) extends this with confidential computation — the server can perform range queries and pattern matching inside a hardware-protected secure enclave without exposing keys.

SQL Code Example

```sql
-- == Always Encrypted key setup (T-SQL metadata only) ==
-- Keys are created via SSMS wizard or PowerShell for production
-- (actual cryptographic material lives outside SQL Server)

-- Column Master Key: metadata pointing to key store
CREATE COLUMN MASTER KEY CMK_AzureKeyVault
WITH (
    KEY_STORE_PROVIDER_NAME = 'AZURE_KEY_VAULT',
    KEY_PATH = 'https://prod-keyvault.vault.azure.net/keys/CMK-SalesDB/ver1'
);

-- Column Encryption Key: encrypted under the CMK
CREATE COLUMN ENCRYPTION KEY CEK_PII
WITH VALUES (
    COLUMN_MASTER_KEY          = CMK_AzureKeyVault,
    ALGORITHM                  = 'RSA_OAEP',
    ENCRYPTED_VALUE            = 0x016E...  -- generated by SSMS wizard
);

-- == Create table with encrypted columns ==
CREATE TABLE HR.EmployeePII (
    EmployeeID   INT IDENTITY PRIMARY KEY,
    DisplayName  NVARCHAR(100)    NOT NULL,  -- plaintext: not sensitive
    NationalID   CHAR(11) ENCRYPTED WITH (   -- deterministic: allows WHERE search
                    ENCRYPTION_TYPE          = DETERMINISTIC,
                    ALGORITHM                = 'AEAD_AES_256_CBC_HMAC_SHA_256',
                    COLUMN_ENCRYPTION_KEY    = CEK_PII),
    AnnualSalary DECIMAL(10,2) ENCRYPTED WITH (  -- randomized: no search needed
                    ENCRYPTION_TYPE          = RANDOMIZED,
                    ALGORITHM                = 'AEAD_AES_256_CBC_HMAC_SHA_256',
                    COLUMN_ENCRYPTION_KEY    = CEK_PII),
    DiagnosisCode NVARCHAR(20) ENCRYPTED WITH (  -- randomized: HIPAA
                    ENCRYPTION_TYPE          = RANDOMIZED,
                    ALGORITHM                = 'AEAD_AES_256_CBC_HMAC_SHA_256',
                    COLUMN_ENCRYPTION_KEY    = CEK_PII)
);

-- Connection string required for client decryption:
-- Server=sql01;Database=SalesDB;Column Encryption Setting=Enabled;
-- Authentication=Active Directory Integrated;
```

Code Walkthrough

The Column Encryption Setting=Enabled connection string parameter activates the driver-side encryption/decryption logic. Without it, the client receives binary ciphertext. The SSMS column encryption wizard handles the key generation and CEK creation step, writing the encrypted key value into SQL Server while the actual CMK material never leaves Azure Key Vault.

3.3 Row-Level Security (RLS)

Row-Level Security enforces fine-grained access control at the row level within a table, transparently to the querying application. RLS uses security predicates — inline table-valued functions — that are automatically appended to every query on the protected table.

Two predicate types are supported. Filter predicates silently remove unauthorised rows from SELECT results; the querying user receives no error and has no indication that filtered rows exist, which is ideal for multi-tenant isolation. Block predicates prevent write operations that would violate the policy: an AFTER INSERT block predicate prevents inserting rows that the user would not be able to read, while a BEFORE UPDATE block predicate prevents updating rows in a way that would move them out of the user's visible set.

RLS is the preferred pattern for multi-tenant SaaS applications where every customer's data lives in the same table and each user should see only their own rows.

SQL Code Example

```sql
-- == Multi-tenant RLS: each user sees only their tenant's data ==

-- Step 1: Create the security predicate function
-- Must be schema-bound; cannot reference non-deterministic functions
CREATE FUNCTION Security.fn_TenantIsolation(@TenantID INT)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN (
    SELECT 1 AS HasAccess
    WHERE
        -- Super-admins bypass isolation
        IS_MEMBER('db_datareader_admin') = 1
        OR
        -- Users see only their assigned tenant
        EXISTS (
            SELECT 1
            FROM dbo.UserTenantMapping utm
            WHERE utm.LoginName  = USER_NAME()
              AND utm.TenantID   = @TenantID
              AND utm.IsActive   = 1
        )
);
GO

-- Step 2: Apply as a security policy on every tenant-scoped table
CREATE SECURITY POLICY Tenancy.TenantPolicy
ADD FILTER PREDICATE Security.fn_TenantIsolation(TenantID)
    ON Sales.Orders,
ADD FILTER PREDICATE Security.fn_TenantIsolation(TenantID)
    ON Sales.Invoices,
ADD BLOCK PREDICATE Security.fn_TenantIsolation(TenantID)
    ON Sales.Orders AFTER INSERT,
ADD BLOCK PREDICATE Security.fn_TenantIsolation(TenantID)
    ON Sales.Orders BEFORE UPDATE
WITH (STATE = ON, SCHEMABINDING = ON);

-- Step 3: Test the isolation
EXECUTE AS USER = 'tenant_42_user';
```

```
SELECT COUNT(*) FROM Sales.Orders;  -- sees only TenantID=42 rows
REVERT;

EXECUTE AS USER = 'tenant_99_user';
SELECT COUNT(*) FROM Sales.Orders;  -- sees only TenantID=99 rows
REVERT;
```

Code Walkthrough

A critical performance consideration: the predicate function is a subquery appended to every query. If the UserTenantMapping table is large and lacks an index on (LoginName, TenantID), RLS can introduce significant overhead. Always create a covering index: CREATE INDEX IX_UTM_Login ON dbo.UserTenantMapping(LoginName) INCLUDE (TenantID, IsActive).

3.4 Dynamic Data Masking & SQL Server Audit

Dynamic Data Masking (DDM) is a lightweight data obfuscation feature that masks column values at query time for users who do not have the UNMASK permission. It does not change the stored data — masked and unmasked users query the same table with the same SQL; the masking layer applies transparently in the result set.

Four built-in mask functions are available. The default() function applies a full mask, displaying zero for numeric columns, a string of X characters for text columns, and 01-01-1900 for date columns. The partial() function reveals a configurable number of leading and trailing characters while masking the middle, making it ideal for national ID numbers and credit card numbers. The email() function preserves the first character and domain structure (for example, aXXX@XXXX.com), giving enough context to confirm a value exists without revealing the actual address. The random() function replaces numeric values with a random number within a specified range, useful for numeric identifiers that must appear plausible but not traceable.

SQL Server Audit provides a comprehensive, tamper-evident audit trail written to files, Windows Event Log, or Application Log. Audits survive database drops and are required for SOX, PCI-DSS, HIPAA, and GDPR Article 30 (Records of Processing Activities) compliance.

SQL Code Example

```
-- ═══ Dynamic Data Masking ═══

-- Apply masks to sensitive columns
ALTER TABLE HR.Employees ALTER COLUMN NationalID CHAR(11)
    MASKED WITH (FUNCTION = 'partial(0,"XXX-XX-",4)');
    -- Stores: 123-45-6789   Shows to masked users: XXX-XX-6789

ALTER TABLE HR.Employees ALTER COLUMN Email NVARCHAR(255)
    MASKED WITH (FUNCTION = 'email()');
    -- Stores: john.doe@company.com   Shows: jXXX@XXXX.com

ALTER TABLE HR.Employees ALTER COLUMN Salary DECIMAL(10,2)
    MASKED WITH (FUNCTION = 'default()');
    -- Shows: 0.00 (full mask for numbers)

```

```
-- Grant UNMASK to HR administrators
GRANT UNMASK ON HR.Employees TO HRAdmin;
GRANT UNMASK TO db_hrmanager_role;  -- role-based

-- Verify masking from a restricted user's perspective
EXECUTE AS USER = 'payroll_temp_user';
SELECT NationalID, Email, Salary FROM HR.Employees WHERE EmployeeID = 101;
-- Returns: XXX-XX-6789 | jXXX@XXXX.com | 0.00
REVERT;

-- == SQL Server Audit ==
USE master;
CREATE SERVER AUDIT ProductionAudit
TO FILE (
    FILEPATH         = 'D:SQLAudit',
    MAXSIZE          = 200 MB,
    MAX_FILES        = 20,
    RESERVE_DISK_SPACE = OFF
)
WITH (
    QUEUE_DELAY  = 1000,   -- 1 second max delay for audit writes
    ON_FAILURE   = CONTINUE  -- or SHUTDOWN to enforce audit integrity
);
ALTER SERVER AUDIT ProductionAudit WITH (STATE = ON);

-- Database-level audit: track all DML on sensitive tables
USE SalesDB;
CREATE DATABASE AUDIT SPECIFICATION Audit_SensitiveAccess
FOR SERVER AUDIT ProductionAudit
ADD (SELECT, INSERT, UPDATE, DELETE ON HR.Employees       BY PUBLIC),
ADD (SELECT, INSERT, UPDATE, DELETE ON Finance.SalaryData BY PUBLIC),
ADD (SELECT ON dbo.CustomerPII                            BY PUBLIC)
WITH (STATE = ON);

-- Query the audit log for compliance reports
SELECT TOP 100
    event_time,
    action_id,       -- SE=SELECT, IN=INSERT, UP=UPDATE, DL=DELETE
    succeeded,
    server_principal_name,
    database_name,
    object_name,
    statement
FROM sys.fn_get_audit_file('D:SQLAuditProductionAudit*.sqlaudit', DEFAULT,
DEFAULT)
WHERE action_id IN ('SE','IN','UP','DL')
  AND object_name IN ('Employees','SalaryData','CustomerPII')
ORDER BY event_time DESC;
```

Code Walkthrough

The ON_FAILURE = SHUTDOWN option is the strictest compliance setting — if the audit file cannot be written (disk full, path unavailable), SQL Server shuts down to prevent unaudited

access. Use CONTINUE for development environments and SHUTDOWN only where regulatory requirements mandate it, as it can cause unexpected outages.

Section 4: High Availability & Cloud Integration

High availability (HA) and disaster recovery (DR) are two distinct but complementary goals. HA minimises unplanned downtime (hardware failure, OS crash, instance failure) through redundancy. DR minimises data loss and recovery time after catastrophic events (site loss, ransomware, regional outage) through geographically distributed copies. Modern architectures increasingly blend on-premises SQL Server HA with Azure cloud services for DR, hybrid analytics, and managed PaaS migrations.

4.1 Always On Availability Groups

Always On Availability Groups (AGs) are SQL Server's premier HA/DR feature. An AG groups one or more databases (an availability group) and replicates them to up to eight secondary replicas. The primary replica handles all read/write traffic. Secondary replicas apply log records asynchronously or synchronously.

Synchronous replicas guarantee zero data loss: a transaction is not committed on the primary until it is hardened on the synchronous secondary. Automatic failover requires synchronous mode.

Asynchronous replicas accept log records without waiting for confirmation, providing higher throughput at the cost of potential data loss on failover. Typically used for geographically distant DR replicas.

The AG Listener is a virtual network name (VNN) that clients connect to — they are transparently redirected to the current primary. With ApplicationIntent=ReadOnly, clients are routed to a read-only secondary for reporting workloads.

◆ RTO vs RPO

With synchronous commit and automatic failover: RTO ≈ 20-30 seconds (WSFC health detection + failover). RPO = 0 (zero data loss). With asynchronous DR replica: RPO depends on redo queue size — monitor log_send_queue_size DMV continuously.

SQL Code Example

```
-- == Always On AG creation (run on primary replica SQL01) ==

-- Pre-requisite: database must be in FULL recovery model
ALTER DATABASE SalesDB SET RECOVERY FULL;

-- Take a full backup (required to initialise AG)
BACKUP DATABASE SalesDB TO DISK = 'D:BackupSalesDB_ForAG.bak' WITH INIT;
BACKUP LOG      SalesDB TO DISK = 'D:BackupSalesDB_ForAG.trn' WITH INIT;

-- Create the Availability Group
CREATE AVAILABILITY GROUP AG_SalesDB
WITH (
    AUTOMATED_BACKUP_PREFERENCE  = SECONDARY,   -- offload backups
```

```
    FAILURE_CONDITION_LEVEL      = 3,            -- auto-failover on SQL errors
    HEALTH_CHECK_TIMEOUT         = 30000,        -- 30s before declaring
unhealthy
    DB_FAILOVER                  = ON            -- fail on DB-level health too
)
FOR DATABASE SalesDB
REPLICA ON
    'SQL01' WITH (
        ENDPOINT_URL              = 'TCP://SQL01.corp.local:5022',
        AVAILABILITY_MODE         = SYNCHRONOUS_COMMIT,
        FAILOVER_MODE             = AUTOMATIC,
        SEEDING_MODE              = AUTOMATIC,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
    ),
    'SQL02' WITH (
        ENDPOINT_URL              = 'TCP://SQL02.corp.local:5022',
        AVAILABILITY_MODE         = SYNCHRONOUS_COMMIT,
        FAILOVER_MODE             = AUTOMATIC,
        SEEDING_MODE              = AUTOMATIC,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
    ),
    'SQL03-DR' WITH (
        ENDPOINT_URL              = 'TCP://SQL03.dr-site.local:5022',
        AVAILABILITY_MODE         = ASYNCHRONOUS_COMMIT,  -- DR site
        FAILOVER_MODE             = MANUAL,
        SEEDING_MODE              = AUTOMATIC,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
    );

-- Add the AG listener (virtual network name for client connections)
ALTER AVAILABILITY GROUP AG_SalesDB
ADD LISTENER 'AG-SalesDB-Listener' (
    WITH IP (('10.0.1.100','255.255.255.0'), ('10.0.2.100','255.255.255.0')),
    PORT = 1433
);

-- Monitor synchronisation health and lag
SELECT
    ag.name,
    ar.replica_server_name,
    ars.role_desc,
    ars.synchronization_health_desc,
    drs.log_send_queue_size / 1024   AS send_queue_mb,   -- RPO indicator
    drs.redo_queue_size / 1024       AS redo_queue_mb,
    drs.last_hardened_time,
    drs.last_redone_time
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar             ON ar.group_id = ag.group_id
JOIN sys.dm_hadr_availability_replica_states ars ON ars.replica_id =
ar.replica_id
JOIN sys.dm_hadr_database_replica_states drs  ON drs.replica_id = ar.replica_id
WHERE drs.is_local = 0;
```

Code Walkthrough

SEEDING_MODE = AUTOMATIC (Direct Seeding) allows SQL Server to automatically seed the secondary database over the network endpoint without requiring a manual backup/restore. This dramatically simplifies initial setup. For large databases over slow WAN links, MANUAL seeding (backup + restore) is still preferred to avoid saturating the link.

4.2 Query Store — The Flight Data Recorder

Query Store was introduced in SQL Server 2016 and is the most important diagnostic tool added to SQL Server in years. It persists query text, execution plans, and runtime statistics to user database tables, surviving server restarts and plan cache flushes. This makes it possible to see exactly what happened to query performance at a specific point in the past.

The three most powerful Query Store use cases are plan regression detection, forced plan stabilisation, and query performance baselining. For plan regression detection, Query Store lets you identify queries whose execution plan changed and degraded after an index change, a statistics update, or a SQL Server version upgrade, because both the old and new plans are retained with their respective runtime statistics. For forced plan stabilisation, you can pin a known-good plan to a specific query using sp_query_store_force_plan, preventing the optimizer from switching to a worse alternative until the underlying root cause is addressed. For baselining, you can compare performance windows before and after any maintenance operation or application deployment to quantify impact with real execution data.

SQL Code Example

```
-- ═══ Configure Query Store ═══
ALTER DATABASE SalesDB SET QUERY_STORE = ON;
ALTER DATABASE SalesDB SET QUERY_STORE (
    OPERATION_MODE              = READ_WRITE,
    CLEANUP_POLICY              = (STALE_QUERY_THRESHOLD_DAYS = 30),
    DATA_FLUSH_INTERVAL_SECONDS = 900,     -- flush to disk every 15 min
    INTERVAL_LENGTH_MINUTES     = 60,      -- aggregation window
    MAX_STORAGE_SIZE_MB         = 2048,    -- 2 GB storage
    QUERY_CAPTURE_MODE          = AUTO,    -- capture significant queries only
    SIZE_BASED_CLEANUP_MODE     = AUTO
);

-- ═══ Find top resource-consuming queries ═══
SELECT TOP 20
    qsq.query_id,
    qsqt.query_sql_text,
    qsp.plan_id,
    qsrs.count_executions,
    ROUND(qsrs.avg_duration        / 1000, 2)  AS avg_ms,
    ROUND(qsrs.avg_logical_io_reads, 0)         AS avg_logical_reads,
    ROUND(qsrs.avg_cpu_time        / 1000, 2)  AS avg_cpu_ms,
    ROUND(qsrs.avg_query_max_used_memory * 8.0 / 1024, 2) AS avg_memory_mb
FROM sys.query_store_query         qsq
JOIN sys.query_store_query_text    qsqt ON qsqt.query_text_id =
qsq.query_text_id
JOIN sys.query_store_plan          qsp  ON qsp.query_id       = qsq.query_id
JOIN sys.query_store_runtime_stats qsrs ON qsrs.plan_id       = qsp.plan_id
JOIN sys.query_store_runtime_stats_interval qsrsi
```

```
    ON qsrsi.runtime_stats_interval_id = qsrs.runtime_stats_interval_id
WHERE qsrsi.start_time >= DATEADD(HOUR,-24,SYSDATETIME())  -- last 24 hours
ORDER BY qsrs.avg_duration DESC;

-- == Find plan regressions: same query, new worse plan ==
SELECT
    qsq.query_id,
    qsqt.query_sql_text,
    regressed.plan_id         AS regressed_plan,
    baseline.plan_id          AS baseline_plan,
    regressed.avg_duration_ms AS regressed_ms,
    baseline.avg_duration_ms  AS baseline_ms,
    regressed.avg_duration_ms / NULLIF(baseline.avg_duration_ms,0) AS
slowdown_factor
FROM sys.query_store_query qsq
JOIN sys.query_store_query_text qsqt ON qsqt.query_text_id = qsq.query_text_id
JOIN (
    SELECT plan_id, query_id, AVG(avg_duration)/1000 AS avg_duration_ms
    FROM sys.query_store_plan qsp
    JOIN sys.query_store_runtime_stats qsrs ON qsrs.plan_id = qsp.plan_id
    GROUP BY plan_id, query_id
) regressed ON regressed.query_id = qsq.query_id
JOIN (
    SELECT plan_id, query_id, AVG(avg_duration)/1000 AS avg_duration_ms
    FROM sys.query_store_plan qsp
    JOIN sys.query_store_runtime_stats qsrs ON qsrs.plan_id = qsp.plan_id
    GROUP BY plan_id, query_id
) baseline ON baseline.query_id = qsq.query_id AND baseline.plan_id <
regressed.plan_id
WHERE regressed.avg_duration_ms > baseline.avg_duration_ms * 2  -- 2x slower
ORDER BY slowdown_factor DESC;

-- == Force the known-good plan ==
EXEC sys.sp_query_store_force_plan @query_id = 42, @plan_id = 7;

-- == Unforce when fixed ==
EXEC sys.sp_query_store_unforce_plan @query_id = 42, @plan_id = 7;
```

Code Walkthrough

sp_query_store_force_plan is a critical emergency tool — when a plan regression causes an outage, you can restore a known-good plan in seconds without code changes, deployments, or plan cache flushes. The force persists across restarts. Always document which plans are forced and why, and unforce them once the root cause (stale statistics, missing index) is resolved.

4.3 Temporal Tables

System-versioned temporal tables automatically maintain a complete history of all row changes. Every INSERT, UPDATE, and DELETE operation automatically records the previous row version in a linked history table, stamped with the valid-time period (ValidFrom, ValidTo). This happens transparently at the storage engine level — no triggers or application changes needed.

Temporal tables enable time-travel queries using FOR SYSTEM_TIME clauses. AS OF returns the state of every row as it existed at a specific point in time, making it ideal for point-in-time audits. BETWEEN … AND returns all row versions that were active at any point within the specified range, inclusive of both endpoints. FROM … TO returns all versions that became active or became inactive within the range, exclusive of the end boundary. CONTAINED IN returns only rows whose entire lifespan falls completely within the specified period, useful for identifying rows created and deleted within a window. Finally, ALL returns every version that has ever existed for each row, including the current state, giving a complete change history.

SQL Code Example

```sql
-- == Create a temporal table ==
CREATE TABLE Pricing.ProductPrices (
    ProductID     INT           NOT NULL,
    ProductName   NVARCHAR(200),
    ListPrice     DECIMAL(10,2),
    CostPrice     DECIMAL(10,2),
    SupplierID    INT,
    -- System-period columns (auto-managed by SQL Server)
    RecordStart   DATETIME2(7) GENERATED ALWAYS AS ROW START NOT NULL,
    RecordEnd     DATETIME2(7) GENERATED ALWAYS AS ROW END   NOT NULL,
    CONSTRAINT PK_ProductPrices PRIMARY KEY (ProductID),
    PERIOD FOR SYSTEM_TIME (RecordStart, RecordEnd)
)
WITH (SYSTEM_VERSIONING = ON (
    HISTORY_TABLE = Pricing.ProductPricesHistory,
    DATA_CONSISTENCY_CHECK = ON
));

-- == Normal DML: history is tracked automatically ==
INSERT INTO Pricing.ProductPrices
(ProductID,ProductName,ListPrice,CostPrice,SupplierID)
VALUES (101,'Widget Pro',19.99,8.50,5);

UPDATE Pricing.ProductPrices SET ListPrice = 24.99 WHERE ProductID = 101;
UPDATE Pricing.ProductPrices SET ListPrice = 29.99 WHERE ProductID = 101;
-- History table now contains both previous versions automatically!

-- == Time-travel queries ==

-- What was the price exactly 3 months ago?
SELECT ProductID, ProductName, ListPrice, RecordStart, RecordEnd
FROM Pricing.ProductPrices
FOR SYSTEM_TIME AS OF DATEADD(MONTH,-3,SYSDATETIME())
WHERE ProductID = 101;

-- Show all price changes for a product (full history)
SELECT ProductID, ProductName, ListPrice,
       RecordStart AS ValidFrom, RecordEnd AS ValidTo
FROM Pricing.ProductPrices
FOR SYSTEM_TIME ALL
WHERE ProductID = 101
ORDER BY RecordStart;
```

```
42
43  -- Rows where price changed within the last 30 days
44  SELECT ProductID, ProductName, ListPrice, RecordStart
45  FROM Pricing.ProductPrices
46  FOR SYSTEM_TIME FROM DATEADD(DAY,-30,SYSDATETIME()) TO SYSDATETIME()
47  WHERE ListPrice <> LAG(ListPrice) OVER (PARTITION BY ProductID ORDER BY
    RecordStart);
```

Code Walkthrough

Temporal tables are an excellent solution for GDPR right-to-erasure (Article 17) audit trails — you can show exactly what data existed for a specific person at any point in time, and when it was modified or deleted. The history table can be stored on a different filegroup with columnstore compression for cost-effective long-term retention.

Section 5: Advanced T-SQL Patterns

Advanced T-SQL goes beyond simple SELECT statements to encompass set-based data transformation, hierarchical data traversal, dynamic query construction, and robust error handling. These patterns distinguish an intermediate SQL developer from a master-level database engineer. Every pattern in this section should be learned to the point of fluency — they appear in interview assessments, code reviews, and production incident responses.

5.1 Window Functions

Window functions compute values across a set of rows related to the current row — the "window" — without collapsing the result set the way GROUP BY does. The OVER() clause defines the window: PARTITION BY divides rows into groups, ORDER BY establishes the ordering within the group, and the ROWS/RANGE frame clause defines which rows relative to the current row are included in the calculation.

Window functions fall into three families. The ranking family includes ROW_NUMBER(), which assigns a unique sequential integer to each row within its partition; RANK(), which assigns the same rank to ties and skips numbers after them; DENSE_RANK(), which assigns the same rank to ties without skipping numbers; and NTILE(n), which divides rows into n roughly equal buckets. The analytic family includes LAG() and LEAD() for accessing values from preceding or following rows, and FIRST_VALUE(), LAST_VALUE(), and NTH_VALUE() for accessing specific positions within the window frame. The aggregate family applies familiar aggregate functions (SUM, AVG, MIN, MAX, COUNT) with an OVER clause, computing rolling totals, moving averages, and running counts without collapsing the result set.

Window functions are evaluated after WHERE, GROUP BY, and HAVING but before ORDER BY — they can reference aliases from SELECT but cannot be used in WHERE. To filter on a window function result, wrap the query in a CTE or subquery.

SQL Code Example

```sql
-- == Running total and 7-day moving average ==
SELECT
    OrderDate,
    DailyRevenue,
    SUM(DailyRevenue)  OVER (ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
    AVG(DailyRevenue)  OVER (ORDER BY OrderDate
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)          AS moving_avg_7day,
    MAX(DailyRevenue)  OVER (ORDER BY OrderDate
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)          AS max_7day
FROM Sales.DailySummary
ORDER BY OrderDate;

-- == LEAD/LAG: compare rows to adjacent rows ==
SELECT
    OrderDate,
    DailyRevenue,
    LAG(DailyRevenue,  1, 0) OVER (ORDER BY OrderDate)  AS prev_day_rev,
    LEAD(DailyRevenue, 1, 0) OVER (ORDER BY OrderDate)  AS next_day_rev,
```

```
    DailyRevenue - LAG(DailyRevenue,1,0)
                            OVER (ORDER BY OrderDate)  AS day_over_day_change,
    ROUND(100.0 * (DailyRevenue - LAG(DailyRevenue,1,0) OVER (ORDER BY
OrderDate))
          / NULLIF(LAG(DailyRevenue,1,0) OVER (ORDER BY OrderDate),0), 2) AS
pct_change
FROM Sales.DailySummary;

-- == NTILE: segment customers into quartiles ==
SELECT
    CustomerID,
    TotalAnnualSpend,
    NTILE(4) OVER (ORDER BY TotalAnnualSpend DESC)  AS spend_quartile,
    CASE NTILE(4) OVER (ORDER BY TotalAnnualSpend DESC)
        WHEN 1 THEN 'Platinum'
        WHEN 2 THEN 'Gold'
        WHEN 3 THEN 'Silver'
        ELSE        'Standard'
    END AS customer_tier
FROM Sales.CustomerAnnualSummary;

-- == Deduplicate: keep only the most recent record per customer ==
WITH RankedOrders AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate DESC)
AS rn
    FROM Sales.Orders
)
SELECT OrderID, CustomerID, OrderDate, TotalAmount
FROM RankedOrders
WHERE rn = 1;  -- most recent order per customer

-- == Running percentage of total ==
SELECT
    ProductCategory,
    CategoryRevenue,
    ROUND(100.0 * CategoryRevenue
          / SUM(CategoryRevenue) OVER (), 2) AS pct_of_total,
    SUM(CategoryRevenue) OVER (ORDER BY CategoryRevenue DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_rev
FROM Sales.CategoryRevenue
ORDER BY CategoryRevenue DESC;
```

Code Walkthrough

The ROWS BETWEEN frame is evaluated in logical row position (based on ORDER BY). RANGE BETWEEN evaluates peers (rows with identical ORDER BY values) together. For financial running totals and moving averages, always use ROWS — it is deterministic and faster. Use RANGE only when you specifically need peer-group semantics.

5.2 Recursive CTEs for Hierarchical Data

Recursive CTEs allow T-SQL to traverse parent-child hierarchical structures: org charts, bill of materials (BOM), category trees, social network friend-of-friend paths, and file system trees. The CTE consists of two parts separated by UNION ALL: the anchor member (non-recursive starting point) and the recursive member (references the CTE by name to produce the next level).

SQL Server limits recursion depth to 100 levels by default. Use OPTION(MAXRECURSION n) to raise or lower the limit (0 = unlimited, use with care).

A key performance consideration is that recursive CTEs process one level at a time and the query engine cannot parallelise them. For very deep hierarchies exceeding 20 levels, or very wide trees with more than 100,000 nodes, the recursive CTE approach can become slow. In those cases, the HierarchyID data type provides a built-in path-based hierarchy with efficient subtree and ancestor queries, while a materialised closure table (a separate table storing all ancestor-descendant pairs) provides the best read performance at the cost of more complex write maintenance.

SQL Code Example

```
-- ═══ Classic example: Organisational hierarchy ═══
WITH OrgChart AS (
    -- Anchor: top-level employees (no manager = C-Suite)
    SELECT
        e.EmployeeID,
        e.FullName,
        e.JobTitle,
        e.ManagerID,
        0                                       AS HierarchyLevel,
        CAST(e.FullName AS NVARCHAR(1000))      AS HierarchyPath,
        CAST(e.EmployeeID AS VARCHAR(500))      AS IDPath
    FROM HR.Employees e
    WHERE e.ManagerID IS NULL

    UNION ALL

    -- Recursive: employees one level below current OrgChart set
    SELECT
        e.EmployeeID,
        e.FullName,
        e.JobTitle,
        e.ManagerID,
        oc.HierarchyLevel + 1,
        CAST(oc.HierarchyPath + ' > ' + e.FullName AS NVARCHAR(1000)),
        CAST(oc.IDPath + '.' + CAST(e.EmployeeID AS VARCHAR(10)) AS
VARCHAR(500))
    FROM HR.Employees e
    JOIN OrgChart oc ON oc.EmployeeID = e.ManagerID
)
SELECT
    REPLICATE('    ', HierarchyLevel) + FullName AS indented_name,
    JobTitle,
    HierarchyLevel,
    HierarchyPath
FROM OrgChart
ORDER BY IDPath
OPTION (MAXRECURSION 15);  -- org depth unlikely to exceed 15
```

```
-- == Bill of Materials: explode all components of a product ==
WITH BOM_Explosion AS (
    -- Anchor: top-level product
    SELECT
        b.ParentProductID,
        b.ChildProductID,
        b.Quantity                      AS DirectQty,
        b.Quantity                      AS ExplodedQty,
        1                               AS Level,
        CAST(b.ChildProductID AS VARCHAR(500)) AS ComponentPath
    FROM Manufacturing.BillOfMaterials b
    WHERE b.ParentProductID = 1001   -- the top product

    UNION ALL

    SELECT
        b.ParentProductID,
        b.ChildProductID,
        b.Quantity,
        bom.ExplodedQty * b.Quantity, -- multiply quantities down the tree
        bom.Level + 1,
        CAST(bom.ComponentPath + '.' + CAST(b.ChildProductID AS VARCHAR) AS
VARCHAR(500))
    FROM Manufacturing.BillOfMaterials b
    JOIN BOM_Explosion bom ON bom.ChildProductID = b.ParentProductID
)
SELECT
    Level,
    ChildProductID,
    p.ProductName,
    DirectQty,
    ExplodedQty,   -- total quantity needed at this level
    ComponentPath
FROM BOM_Explosion boe
JOIN Products p ON p.ProductID = boe.ChildProductID
ORDER BY ComponentPath;
```

Code Walkthrough

The IDPath column (concatenated integer IDs) provides a reliable sort order for displaying the hierarchy in tree order. Sorting by HierarchyPath (name-based) can break when names are not alphabetically ordered within their level. The IDPath approach mirrors the HierarchyID type's path-based ordering without its complexity.

5.3 Dynamic SQL & sp_executesql

Dynamic SQL — queries constructed as strings at runtime — is necessary when column names, table names, or the overall query structure must vary based on input. However, it introduces two serious risks: SQL injection and poor plan reuse.

SQL injection: if you concatenate user-supplied strings directly into a SQL statement, a malicious user can inject arbitrary SQL. Always use sp_executesql with parameterised values for all user-supplied data values.

Plan reuse: EXEC('string') generates a new plan every execution because the exact string differs. sp_executesql with a stable query template and varying parameters allows SQL Server to cache and reuse the plan.

The golden rule of dynamic SQL is to always parameterise data values and always whitelist object names. Data values supplied by callers must be passed as typed parameters through sp_executesql, never concatenated into the query string. Object names such as table names and column names cannot be parameterised, so they must be validated against a known-safe list from a system catalogue before being included, and always wrapped in QUOTENAME() before interpolation.

SQL Code Example

```sql
-- ❌ VULNERABLE: direct string concatenation — NEVER DO THIS
DECLARE @TableName NVARCHAR(100) = 'Orders; DROP TABLE Orders; --';
EXEC ('SELECT * FROM ' + @TableName);  -- catastrophic injection!

-- ✅ SAFE PATTERN 1: parameterise the value, validate the object name
DECLARE @StatusFilter NVARCHAR(50) = 'Active';
DECLARE @sql NVARCHAR(500) = N'SELECT OrderID, TotalAmount
    FROM Sales.Orders WHERE Status = @Status';

EXEC sp_executesql @sql,
    N'@Status NVARCHAR(50)',   -- parameter type declaration
    @Status = @StatusFilter;  -- parameter value binding

-- ✅ SAFE PATTERN 2: whitelist-validate object name (never trust user input)
DECLARE @TableInput NVARCHAR(128) = 'Orders';
DECLARE @SafeTable  NVARCHAR(128);

-- Only allow known, expected table names
SELECT @SafeTable = name
FROM sys.tables
WHERE name = @TableInput AND SCHEMA_NAME(schema_id) = 'Sales';

IF @SafeTable IS NULL
    THROW 50001, 'Invalid table name.', 1;

DECLARE @dynSQL NVARCHAR(500) = N'SELECT COUNT(*) FROM Sales.' +
QUOTENAME(@SafeTable);
EXEC sp_executesql @dynSQL;  -- QUOTENAME adds [] to prevent injection

-- ✅ Dynamic PIVOT: generate column list from data
DECLARE @PivotCols NVARCHAR(MAX),
        @PivotSQL  NVARCHAR(MAX);

-- Build quoted column list from actual year data
SELECT @PivotCols = STRING_AGG(QUOTENAME(yr), ', ')
    WITHIN GROUP (ORDER BY yr)
```

```
FROM (SELECT DISTINCT YEAR(OrderDate) AS yr FROM Sales.Orders) y;

SET @PivotSQL = N'
SELECT CustomerID, ' + @PivotCols + N'
FROM (
    SELECT CustomerID, YEAR(OrderDate) AS yr, TotalAmount
    FROM Sales.Orders
) src
PIVOT (SUM(TotalAmount) FOR yr IN (' + @PivotCols + N')) pvt
ORDER BY CustomerID;';

EXEC sp_executesql @PivotSQL;
```

Code Walkthrough

QUOTENAME() is your defence against object-name injection — it wraps a name in square brackets and escapes any embedded brackets, ensuring the string is always interpreted as an identifier rather than SQL syntax. Always use QUOTENAME on any object name that comes from user input or a dynamic lookup, even when you believe the source is safe.

5.4 Robust Error Handling & Transaction Management

Production-grade stored procedures must handle errors explicitly. The TRY/CATCH construct catches runtime errors (but not compile-time errors or severity 20+ fatal errors). Within CATCH, the XACT_STATE() function reports the transaction state: 1 = committable, -1 = doomed (cannot commit), 0 = no transaction.

SET XACT_ABORT ON automatically rolls back and aborts the batch on any run-time error — it is simpler than manual XACT_STATE checks and is the recommended default for stored procedures. Combine it with an explicit TRY/CATCH for logging and re-raising.

Always log errors to a dedicated error table rather than returning them to the application immediately — this creates a persistent record for post-incident analysis.

SQL Code Example

```
-- == Production-grade stored procedure pattern ==
CREATE PROCEDURE Sales.usp_ProcessOrderTransfer
    @OrderID        INT,
    @NewCustomerID  INT,
    @ProcessedBy    NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;  -- auto-rollback on any error; no partial commits

    -- Input validation (before opening transaction)
    IF @OrderID IS NULL OR @NewCustomerID IS NULL
        THROW 50001, 'OrderID and NewCustomerID are required parameters.', 1;

    IF NOT EXISTS (SELECT 1 FROM Sales.Customers WHERE CustomerID =
@NewCustomerID AND IsActive=1)
        THROW 50002, 'Target customer does not exist or is inactive.', 1;
```

```
17
18      BEGIN TRANSACTION;
19      BEGIN TRY
20
21          -- Verify order is in a transferable state (pessimistic locking)
22          DECLARE @CurrentStatus NVARCHAR(50);
23          SELECT @CurrentStatus = Status
24          FROM Sales.Orders WITH (UPDLOCK, ROWLOCK)  -- lock the row
25          WHERE OrderID = @OrderID;
26
27          IF @CurrentStatus IS NULL
28              THROW 50003, 'Order not found.', 1;
29
30          IF @CurrentStatus NOT IN ('Pending','Active')
31              THROW 50004, 'Order cannot be transferred in its current status.',
    1;
32
33          -- Execute the transfer
34          UPDATE Sales.Orders
35          SET CustomerID   = @NewCustomerID,
36              ModifiedDate = SYSDATETIME(),
37              ModifiedBy   = @ProcessedBy
38          WHERE OrderID = @OrderID;
39
40          -- Write to audit trail
41          INSERT INTO Audit.OrderTransfers
42              (OrderID, NewCustomerID, TransferredAt, TransferredBy,
    PreviousStatus)
43          VALUES
44              (@OrderID, @NewCustomerID, SYSDATETIME(), @ProcessedBy,
    @CurrentStatus);
45
46          COMMIT TRANSACTION;
47          SELECT 'SUCCESS' AS Result, @OrderID AS OrderID;
48
49      END TRY
50      BEGIN CATCH
51          -- Rollback if there is an active (possibly doomed) transaction
52          IF XACT_STATE() <> 0
53              ROLLBACK TRANSACTION;
54
55          -- Log to persistent error table for audit
56          INSERT INTO dbo.ProcedureErrorLog
57              (ProcedureName, ErrorNumber, ErrorMessage, ErrorLine, ErrorTime,
    Parameters)
58          VALUES (
59              'Sales.usp_ProcessOrderTransfer',
60              ERROR_NUMBER(), ERROR_MESSAGE(), ERROR_LINE(), SYSDATETIME(),
61              CONCAT('OrderID=',@OrderID,', NewCustomerID=',@NewCustomerID)
62          );
63
64          -- Re-raise to the caller with full context
65          THROW;
```

```
66      END CATCH;
67  END;
```

Code Walkthrough

The UPDLOCK + ROWLOCK combination in the SELECT statement is the correct pattern for read-then-update operations: it acquires an update lock immediately during the read, preventing another session from modifying the row between your read and your update (the "lost update" race condition). Without this hint, two concurrent transfer requests for the same order could both pass the status check and both execute, resulting in corruption.

Appendix A: Quick Reference — Key DMVs & System Views

The following table summarises the most frequently used DMVs and system views for each major topic area covered in CSD624. Use this as a revision aid and production reference.

Category	DMV / View	Purpose
Execution Plans	sys.dm_exec_query_stats	Cached query runtime statistics: reads, CPU, elapsed time
Execution Plans	sys.dm_exec_query_plan	Retrieve execution plan XML by plan handle
Execution Plans	sys.dm_exec_sql_text	Retrieve query text by SQL handle
Statistics	sys.stats	All statistics objects on user tables
Statistics	sys.dm_db_stats_properties	Last update, sample rate, row count per statistics object
Indexes	sys.indexes	All index definitions: type, key columns, fill factor
Indexes	sys.index_columns	Column membership of each index
Indexes	sys.dm_db_index_physical_stats	Fragmentation, page count, depth per index
Indexes	sys.dm_db_index_usage_stats	Seeks, scans, lookups, updates since last restart
Indexes	sys.dm_db_missing_index_details	Missing index suggestions with column breakdown
Security	sys.database_permissions	All permission assignments in the current database
Security	sys.fn_get_audit_file	Read SQL Server Audit log files
Security	sys.dm_database_encryption_keys	TDE encryption state and progress
HA / AG	sys.availability_groups	AG names and configuration
HA / AG	sys.availability_replicas	Replica endpoints and modes
HA / AG	sys.dm_hadr_database_replica_states	Real-time sync state, queue sizes, last hardened time
Query Store	sys.query_store_query	Stored query fingerprints
Query Store	sys.query_store_plan	Stored execution plans per query
Query Store	sys.query_store_runtime_stats	Aggregated runtime metrics per plan
Waits	sys.dm_os_wait_stats	Cumulative wait type statistics since restart
Sessions	sys.dm_exec_sessions	Active sessions: login, host, CPU, reads

Appendix B: T-SQL Pattern Cheat Sheet

Pagination with OFFSET/FETCH

```sql
-- Page N of PageSize rows (page 1 = first page)
DECLARE @Page INT = 2, @PageSize INT = 25;
SELECT OrderID, CustomerID, OrderDate, TotalAmount
FROM Sales.Orders
ORDER BY OrderDate DESC
OFFSET (@Page - 1) * @PageSize ROWS
FETCH NEXT @PageSize ROWS ONLY;
```

Upsert with MERGE

```sql
-- Insert new rows; update existing rows based on key
MERGE Sales.CustomerStats AS target
USING (
    SELECT CustomerID, SUM(TotalAmount) AS TotalSpend, COUNT(*) AS OrderCount
    FROM Sales.Orders
    WHERE OrderDate >= '2024-01-01'
    GROUP BY CustomerID
) AS source
ON target.CustomerID = source.CustomerID
WHEN MATCHED THEN
    UPDATE SET TotalSpend  = source.TotalSpend,
               OrderCount  = source.OrderCount,
               LastUpdated = SYSDATETIME()
WHEN NOT MATCHED BY TARGET THEN
    INSERT (CustomerID, TotalSpend, OrderCount, LastUpdated)
    VALUES (source.CustomerID, source.TotalSpend, source.OrderCount,
SYSDATETIME())
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;  -- remove customers with no orders in period
```

String Aggregation (STRING_AGG)

```sql
-- Concatenate values per group (SQL Server 2017+)
SELECT
    CustomerID,
    STRING_AGG(ProductName, ', ')
        WITHIN GROUP (ORDER BY OrderDate DESC) AS products_ordered
FROM Sales.OrderItems oi
JOIN Products p ON p.ProductID = oi.ProductID
GROUP BY CustomerID;
```

JSON Output

```sql
-- Return query results as JSON (SQL Server 2016+)
SELECT CustomerID, CustomerName, Email, TotalSpend
FROM Sales.CustomerSummary
WHERE TotalSpend > 10000
ORDER BY TotalSpend DESC
FOR JSON PATH, ROOT('TopCustomers');
```

```
7
8  -- Parse incoming JSON
9  SELECT j.CustomerID, j.OrderDate, j.Total
10 FROM OPENJSON(@InputJson, '$.orders')
11 WITH (
12     CustomerID INT          '$.customer_id',
13     OrderDate  DATE         '$.order_date',
14     Total      DECIMAL(12,2) '$.total_amount'
15 ) j;
```